PLATAFORMA DE GESTIÓN COMERCIAL

Documento de análisis funcional y modelo de datos para desarrollo

1. Enunciado del problema

Una organización comercial requiere una plataforma web para administrar clientes, rutas comerciales, visitas, seguimientos, cotizaciones y pedidos. Actualmente la información se gestiona de manera dispersa en excel, dificultando el control de la operación comercial, el seguimiento a clientes y la medición de indicadores.

Como equipo de desarrollo deberán diseñar y desarrollar una solución tipo Gestor Comercial que permita planear rutas, registrar visitas, gestionar oportunidades de negocio y generar indicadores para la toma de decisiones.

Solo se podrá utilizar arquitectura Hexagonal o Clean Architecture

2. Objetivo General

Desarrollar una plataforma integral de gestión comercial que centralice la administración de clientes, actividades comerciales, visitas, cotizaciones y pedidos.

3. Alcance Funcional

- Administración de clientes y contactos
- Gestión de asesores comerciales
- Planeación de rutas
- Agenda y registro de visitas
- Seguimiento comercial
- Gestión de cotizaciones
- Dashboard con KPI comerciales
- Reportes e indicadores

4. Modelo Entidad Relación Propuesto

CLIENTE

- id_cliente PK
- nit
- razon_social
- nombre_comercial
- direccion
- departamento
- municipio
- barrio
- telefono
- email
- id_zona FK
- id_tipo_cliente FK
- id_potencial FK
- id_estado_cliente FK
- frecuencia_visita_dias
- fecha_creacion
- activo

CONTACTO_CLIENTE

- id_contacto PK
- id_cliente FK
- nombre
- cargo
- telefono
- celular
- email
- principal

ASESOR

- id_asesor PK
- nombre
- correo

- telefono
- estado

ZONA

- id_zona PK
- nombre

TIPO_CLIENTE

- id_tipo_cliente PK
- nombre

POTENCIAL

- id_potencial PK
- nombre
- orden

ESTADO_CLIENTE

- id_estado_cliente PK
- nombre

PORTAFOLIO

- id_portafolio PK
- nombre
- descripcion

CLIENTE_PORTAFOLIO

- id_cliente FK
- id_portafolio FK
- fecha_registro

RUTA

- id_ruta PK
- nombre
- id_asesor FK
- fecha_inicio
- fecha_fin

- estado

RUTA_CLIENTE

- id_ruta_cliente PK
- id_ruta FK
- id_cliente FK
- orden_visita

VISITA

- id_visita PK
- id_cliente FK
- id_asesor FK
- id_ruta FK
- fecha_programada
- fecha_real
- estado_visita

RESULTADO_VISITA

- id_resultado PK
- nombre

VISITA_RESULTADO

- id_visita_resultado PK
- id_visita FK
- id_resultado FK
- observacion

SEGUIMIENTO

- id_seguimiento PK
- id_visita FK
- fecha_proxima
- actividad
- observacion
- completado

COTIZACION

- id_cotizacion PK

- numero
- id_cliente FK
- id_asesor FK
- fecha
- valor
- estado

COTIZACION_DETALLE

- id_detalle PK
- id_cotizacion FK
- producto
- cantidad
- precio
- subtotal

PEDIDO

- id_pedido PK
- numero
- id_cliente FK
- id_cotizacion FK
- fecha
- valor
- estado

HISTORIAL_CLIENTE

- id_historial PK
- id_cliente FK
- fecha
- accion
- usuario
- descripcion

5. Relaciones Principales

- ASESOR 1:N RUTA
- ASESOR 1:N VISITA
- CLIENTE 1:N CONTACTO_CLIENTE
- CLIENTE N:M PORTAFOLIO

- CLIENTE N:1 ZONA
- CLIENTE N:1 TIPO_CLIENTE
- CLIENTE N:1 POTENCIAL
- CLIENTE N:1 ESTADO_CLIENTE
- RUTA 1:N RUTA_CLIENTE
- CLIENTE 1:N VISITA
- VISITA 1:N VISITA_RESULTADO
- VISITA 1:N SEGUIMIENTO
- CLIENTE 1:N COTIZACION
- COTIZACION 1:N COTIZACION_DETALLE
- CLIENTE 1:N PEDIDO

6. Requerimientos Mínimos

- Autenticación de usuarios mediante betterAuth
- Dashboard de indicadores
- Mapa de visitas, mediante Open Street Maps
- Agenda commercial para los asesores
- Gestión documental de cotizaciones (ingreso de cotización simulada)
- Bitácora de actividades
- Exportación de reports mediante CSV
- Utilizar importaciones mediante TypeORM

Distribución de equipos de trabajo – Ruta Avanzada Node.js

Para el desarrollo de la actividad de construcción de una aplicación con backend en NestJS y frontend, se organizará a los participantes en cuatro equipos de trabajo, cada uno liderado por un líder seleccionado por el tutor de la ruta.

1. Estructura de supervisión

- Team Leader: recibe los avances y resultados directamente del tutor.
- Tutor de la ruta: selecciona a cuatro participantes como líderes de equipo, supervisa su gestión y recibe los reportes de cada uno. No hará parte de ningún equipo de desarrollo.
- Líderes de equipo: cada uno será responsable de organizar, orientar y hacer seguimiento al trabajo de sus integrantes, así como de reportar los avances al tutor.
- Desarrolladores: desarrollarán las tareas asignadas para la construcción de la aplicación, trabajando colaborativamente en el backend y el frontend.

1. Distribución de participantes

La ruta cuenta con 31 coders en total: 1 tutor y 30 participantes. El tutor no formará parte de ningún equipo de desarrollo. De los 30 participantes, cuatro serán líderes y los otros 26 se distribuirán entre los equipos.

Equipo	Líder	Desarrolladores	Total por equipo
Equipo 1	1	6	7
Equipo 2	1	6	7
Equipo 3	1	7	8
Equipo 4	1	7	8
Total	4	26	30

Tutor: responsable de supervisar a los cuatro líderes.

Sistema de puntuación y sustentación de proyectos

1. Sistema de puntuación

El equipo que presente la **mejor solución propuesta, implementada y sustentada** podrá obtener **20 puntos adicionales en la prueba de desempeño**.

La asignación de estos puntos será exclusiva para un único equipo ganador. Para determinarlo, se evaluará la calidad de la solución desarrollada, el cumplimiento de los requisitos técnicos, la implementación funcional, la calidad de la sustentación y la capacidad de los integrantes para demostrar y explicar el funcionamiento de su proyecto.

2. Fecha y condiciones de la sustentación

- **Fecha:** lunes 26 de octubre de 2026.
- **Duración máxima:** una hora por equipo.
- **Asistencia obligatoria:** todos los integrantes del equipo deberán estar presentes durante la sustentación.
- **Modalidad:** demostración práctica de la aplicación desplegada, explicación de la arquitectura, presentación de las funcionalidades implementadas y sustentación de las decisiones técnicas.

Cada equipo deberá administrar el tiempo disponible para presentar su solución, demostrar su funcionamiento y responder las preguntas que se formulen durante la evaluación.

3. Requisitos técnicos obligatorios

Cada equipo deberá entregar una solución funcional, desplegada en AWS y disponible para su demostración durante la sustentación. La arquitectura deberá cumplir con los siguientes requisitos:

3.1. Aplicación desplegada en AWS

- La aplicación deberá estar desplegada en una instancia EC2 de AWS.
- La solución deberá contar con un backend desarrollado con NestJS y un frontend funcional.
- La aplicación deberá estar disponible para su validación y demostración durante la sustentación.

3.2. Base de datos PostgreSQL en RDS

- La persistencia de la información deberá realizarse mediante una instancia Amazon RDS con PostgreSQL.
- La aplicación deberá conectarse a la base de datos desplegada en AWS.
- La información deberá almacenarse y consultarse correctamente desde la aplicación.

3.3. Almacenamiento de archivos multimedia en S3

- Todas las imágenes, documentos y demás archivos multimedia utilizados por la aplicación deberán almacenarse en Amazon S3.
- Ningún archivo multimedia deberá estar almacenado dentro del código fuente del proyecto ni incorporado al repositorio de GitHub.
- La base de datos deberá almacenar la referencia que permita identificar y recuperar cada objeto almacenado en S3, asociándolo con el registro correspondiente de la aplicación.
- La aplicación deberá permitir el acceso o la visualización de los archivos según las funcionalidades implementadas y los permisos definidos.

3.4. Automatización de despliegues con Jenkins

- Cada equipo deberá instalar y configurar Jenkins en la misma instancia EC2 donde se despliegue la aplicación.
- Jenkins deberá permitir ejecutar el proceso de despliegue de la aplicación.
- El equipo deberá demostrar que puede desplegar una nueva versión o aplicar cambios mediante el proceso automatizado configurado en Jenkins.
- El proceso deberá estar integrado con el repositorio de GitHub.

3.5. Repositorio de código en GitHub

- El código fuente del proyecto deberá estar alojado en GitHub.
- El repositorio deberá contener el código necesario para construir y ejecutar la aplicación, así como los archivos de configuración y documentación pertinentes.
- No deberán incluirse archivos multimedia, credenciales, contraseñas, claves privadas ni secretos de configuración.
- Las variables de entorno y credenciales deberán gestionarse de forma segura, sin publicarse en el repositorio.

4. Criterios para determinar el equipo ganador

La evaluación tendrá en cuenta los siguientes aspectos:

1. **Calidad de la solución propuesta:** pertinencia, claridad y valor de la solución frente a la necesidad que busca resolver.
2. **Implementación funcional:** cumplimiento de los objetivos, funcionamiento de las características desarrolladas y calidad de la experiencia de usuario.
3. **Arquitectura y despliegue:** correcta integración de EC2, RDS PostgreSQL, S3 y Jenkins.
4. **Automatización y control de versiones:** organización del repositorio y demostración del despliegue y actualización de la aplicación mediante Jenkins.
5. **Sustentación técnica:** claridad de la presentación, dominio de la solución, justificación de las decisiones técnicas y capacidad para responder preguntas.
6. **Participación del equipo:** conocimiento demostrable del proyecto por parte de sus integrantes y capacidad para explicar sus aportes.

El equipo ganador será aquel que demuestre la mejor combinación de propuesta, implementación funcional, cumplimiento técnico y sustentación.

5. Consideraciones finales

Los requisitos técnicos descritos son condiciones obligatorias para la presentación del proyecto. Cada equipo deberá verificar previamente que todos los componentes estén desplegados, configurados y funcionando correctamente.

Se recomienda realizar pruebas de extremo a extremo antes de la sustentación, incluyendo la ejecución del frontend y backend, la conexión con PostgreSQL, la carga y consulta de archivos en S3 y el despliegue de cambios desde Jenkins.

El objetivo no es únicamente presentar una aplicación funcional, sino demostrar la capacidad del equipo para construir, desplegar, mantener y sustentar una solución de software utilizando buenas prácticas de desarrollo y servicios de infraestructura en la nube.